

Resume Analyzer and Job Matcher

Final Project Report

CAI4002: Introduction to Artificial Intelligence

Team: Martin Dang, Tuan Khang Phan

Summary. This report describes the final state of the Resume Analyzer and Job Matcher: an AI system that extracts a confidence-weighted skill profile from a resume (PDF, DOCX, or plain text), parses a job posting into required and preferred qualifications, computes a blended TF-IDF, skill-coverage and Sentence-BERT compatibility score, classifies the resume's job category with a Naive Bayes model, and recommends the edits that would raise the score the most, all reachable from a minimal web interface.

2,484	24	244	68.6%
resumes in corpus	job categories	skills in taxonomy	best classifier accuracy

1. Project Status

The system is feature-complete. Every pipeline stage from the original proposal is implemented and verified end to end: resume ingestion (PDF, DOCX, TXT), confidence-weighted skill extraction over a 244-skill taxonomy spanning all 24 dataset job categories, job-posting constraint parsing, a three-signal compatibility score (TF-IDF corpus percentile, skill coverage, and pre-trained Sentence-BERT semantic similarity), a greedy recommendation engine, a Naive Bayes category classifier, and a Streamlit web interface tying all of it together for an end user.

All code, experiment scripts, figures, and results referenced in this report are versioned in the team's GitHub repository (github.com/khangpt2k6/Intro_to_AI).

2. Dataset

The system uses the public Resume Dataset from Kaggle ([snehaanbhawal/resume-dataset](https://www.kaggle.com/snehaanbhawal/resume-dataset)): 2,484 real resumes collected from [livecareer.com](https://www.livecareer.com), labeled into 24 job categories. The dataset trains the TF-IDF vector space and the category classifier, and supplies realistic test resumes for the demo. A public dataset keeps the project reproducible and avoids the privacy concerns of collecting real applicant data.

Split	Size
-------	------

Total resumes	2,484
Job categories	24
Training split (80%)	1,987 resumes
Held-out test split (20%)	497 resumes

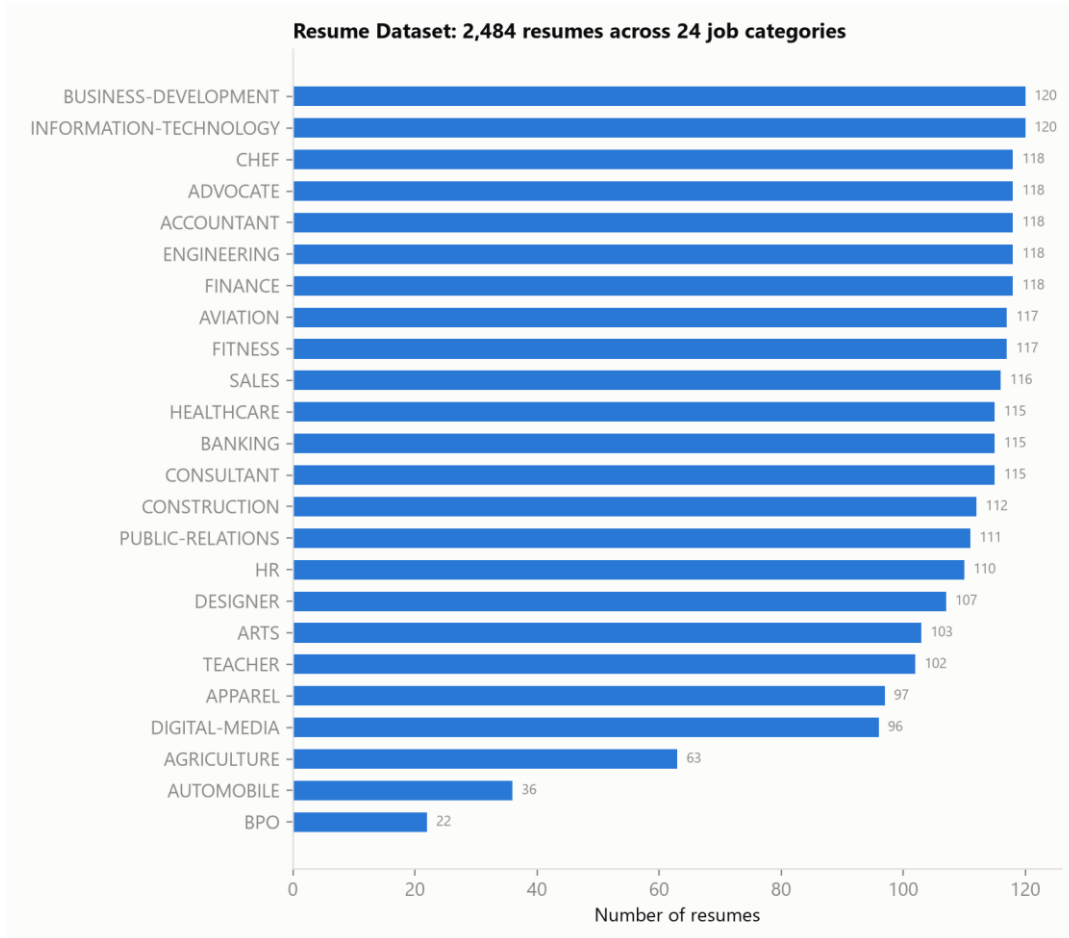


Figure 1. Distribution of the 2,484 resumes across the 24 job categories. The class imbalance (120 vs. 22 resumes) is one challenge the classifier has to handle.

3. System Architecture

All six pipeline stages are implemented and working:

Stage	Implementation
Resume ingestion	load_resume() extracts text from .pdf (pypdf), .docx (python-docx), or .txt, then normalizes whitespace and unicode punctuation (smart quotes, bullets, dashes) so extraction behaves the same regardless of source format.

Skill extraction and confidence	A 244-skill taxonomy (685 aliases) matched with word-bounded regular expressions. Each extracted skill gets a confidence in [0, 1] blending mention frequency, experience context (proximity to action words like 'built' or 'deployed'), and Naive Bayes category agreement.
Job description parsing	Postings are split into hard (required) and soft (preferred) constraints.
Compatibility scoring	Score = 0.4 x confidence-weighted skill coverage + 0.3 x corpus-percentile TF-IDF cosine + 0.3 x pre-trained Sentence-BERT cosine similarity.
Recommendation engine	Missing skills ranked greedy best-first by exact score gain recovered; required skills carry twice the weight of preferred ones.
Category classification	Naive Bayes and Logistic Regression over TF-IDF features, evaluated on a stratified held-out test set.

4. Skill Confidence Model

Binary alias matching tells us a skill is mentioned; it cannot tell us whether the resume genuinely demonstrates the skill or just lists it. A small Multinomial Naive Bayes classifier estimates $P(\text{category} \mid \text{resume text})$ over the 24 dataset categories, and each skill in the taxonomy is assigned an expected category profile from that same model (its predicted-category distribution over a pseudo-document made of the skill's own alias phrases). A skill whose home categories match the resume's predicted category, for example TensorFlow on a resume that reads as Information Technology, is trusted more than the same skill appearing on an unrelated resume, such as TensorFlow on an accountant resume.

Confidence blends three signals with weights 0.40 (mention frequency), 0.25 (experience context), and 0.35 (Naive Bayes category agreement). Without a classifier supplied, `extract_skills()` falls back to frequency and context only, so it still runs as a pure text function with no model or dataset dependency. The matcher credits each matched job requirement by the resume's confidence in that skill, so a genuinely demonstrated skill contributes more to the coverage score than a name-dropped one.

5. Semantic Similarity

TF-IDF only rewards lexical overlap: a resume that says 'built REST services in Django' gets little credit against a posting asking for 'web backend development in Python', even though the two phrases describe the same capability. To capture that kind of paraphrase, the matcher adds a third signal: cosine similarity between sentence embeddings from a pre-trained Sentence-BERT

model (all-MiniLM-L6-v2). On the demo the signal tracks fit closely, from 0.35 for the accountant posting to 0.68 for the software engineer posting.

Scope note. This model is used strictly as a pre-trained black box for embeddings. It is loaded once, used only to encode text into vectors, and is never fine-tuned or otherwise updated on this project's data, consistent with the course's scope, which covers basic deep learning concepts but not model training at the transformer scale.

6. Classifier Experiments

Two classifiers were trained to predict a resume's job category from its text, using an 80/20 stratified train/test split (1,987 training, 497 test resumes). With 24 classes, random guessing would score about 4.2%.

Model	Accuracy	Macro F1
Multinomial Naive Bayes	57.3%	0.494
Logistic Regression	68.6%	0.637

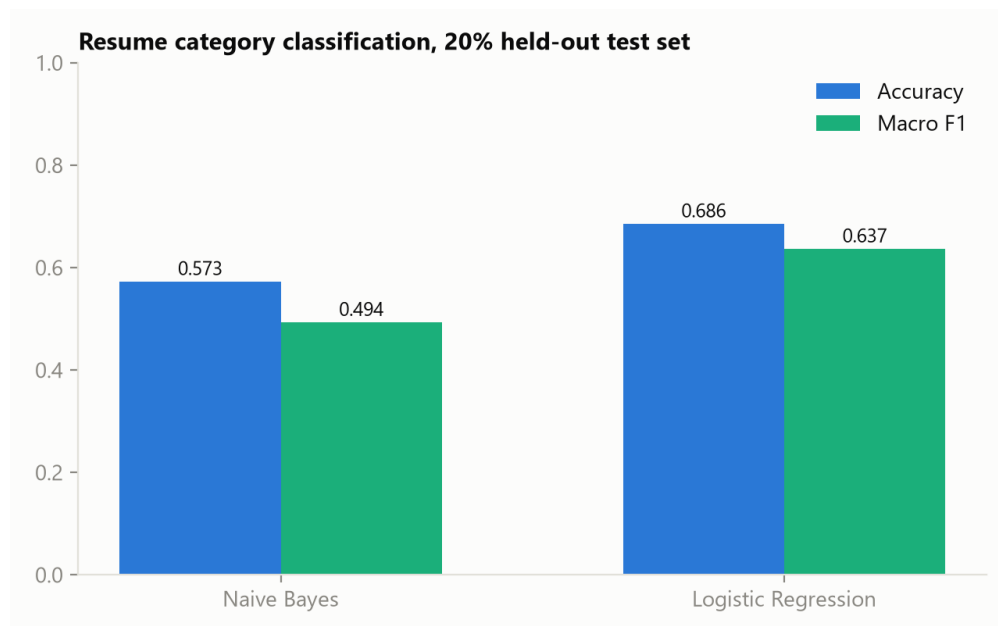


Figure 2. Both models beat the 4.2% random baseline by a wide margin; Logistic Regression is the stronger of the two.

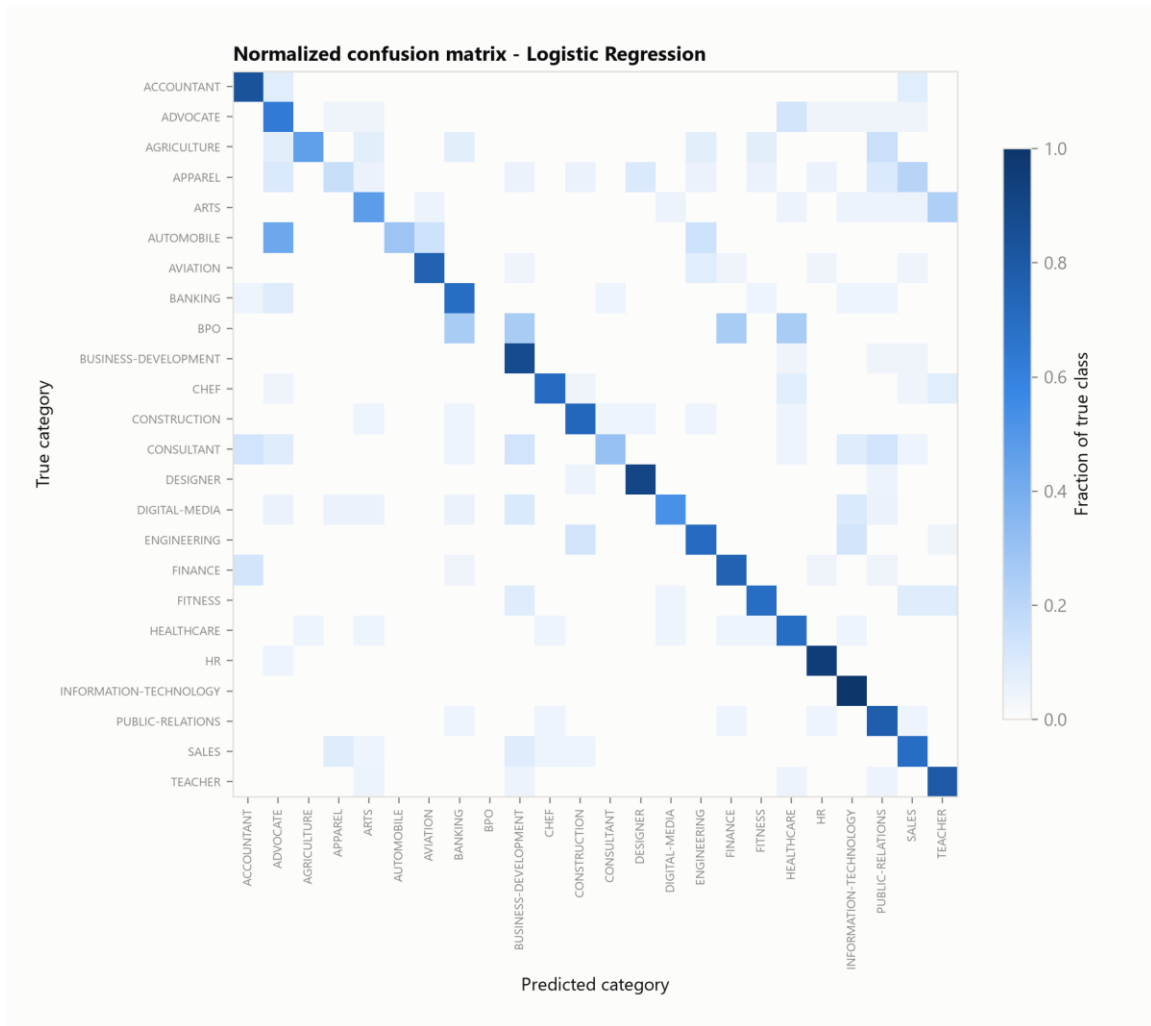


Figure 3. Normalized confusion matrix for Logistic Regression. Remaining confusion concentrates in genuinely overlapping categories, for example Finance vs. Accountant and Sales vs. Business Development.

Note on reproducibility. The train/test split uses a fixed random_state, and Naive Bayes reproduces exactly run to run (57.34% accuracy, 0.494 macro F1). Logistic Regression varies by well under one point between runs of the same script (68.61% accuracy and 0.637 macro F1 in the run reported here), which comes from floating-point non-determinism in scikit-learn's solver rather than from any change to the taxonomy or features, since run_experiments.py does not depend on either.

7. End-to-End Demo

To validate the whole pipeline, one real Information Technology resume from the dataset (ID 83816738) was matched against three job postings written in realistic formats: a backend software engineer role, a data analyst role, and a staff accountant role. The system behaves the way a human reviewer would expect: the IT resume scores 71.6/100 against the software

engineer posting, 47.4/100 against the data analyst posting, and only 13.5/100 against the accountant posting, a clear and well-separated ranking that matches the resume's actual skill set.

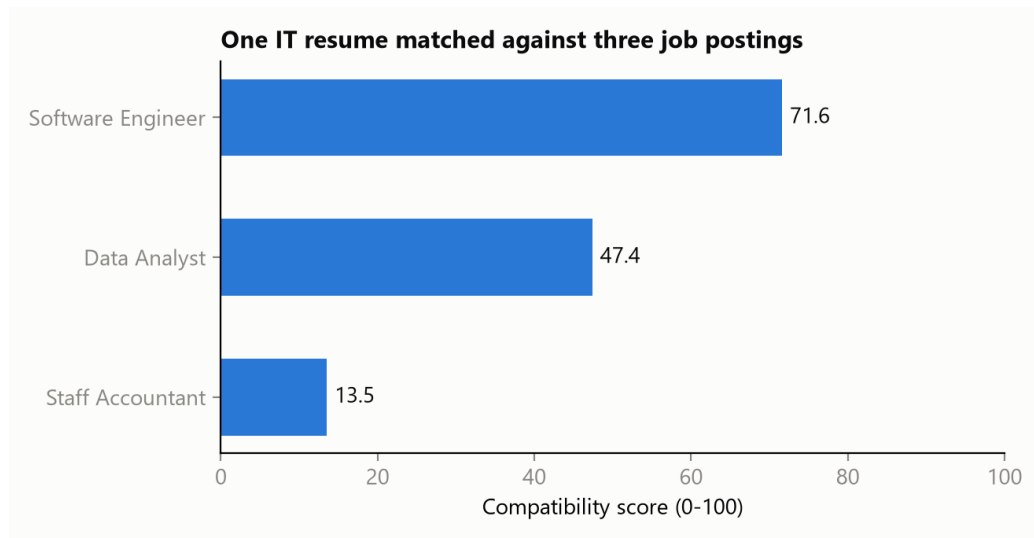


Figure 4. The same resume scored against three postings. The score cleanly separates the strong fit from the partial and poor fits.

For the software engineer posting, the analyzer matched 6 of 8 required skills (AWS, Agile, Git, Java, REST APIs, SQL) plus 3 preferred skills (CI/CD, Linux, React), and identified Docker, Python as the top missing required skills, each worth an estimated +3.8 points.

Posting	Score	Skill coverage	Semantic (SBERT)
Software Engineer - Backend	71.6/100	53%	0.68
Data Analyst	47.4/100	14%	0.44
Staff Accountant	13.5/100	0%	0.35

```

RESUME ANALYZER AND JOB MATCHER - DEMO
Test resume: dataset ID 83816738 (true category: INFORMATION-TECHNOLOGY)
=====

Top extracted skills (confidence = freq + context + NB agreement)
SQL          conf 0.91   x10, context 0.65, agreement 0.99
PHP          conf 0.88   x4, context 1.00, agreement 0.65
Databases    conf 0.87   x6, context 0.65, agreement 0.88
Spring       conf 0.86   x3, context 1.00, agreement 0.60
Bootstrap    conf 0.82   x3, context 0.77, agreement 0.64
Linux        conf 0.82   x2, context 0.65, agreement 0.97
Java         conf 0.81   x15, context 0.91, agreement 0.52
HTML/CSS     conf 0.80   x7, context 0.80, agreement 0.57
Git          conf 0.80   x9, context 0.61, agreement 0.69
Project Management conf 0.78   x2, context 1.00, agreement 0.60
CI/CD        conf 0.76   x4, context 0.82, agreement 0.45
Microservices conf 0.75   x3, context 0.77, agreement 0.46

JOB: Staff Accountant
Compatibility score : 13.5/100
Skill coverage      : 0% (cosine 0.009, beats 10% of corpus)
Semantic similarity : 0.348 (pre-trained Sentence-BERT embedding cosine)
Required matched    : (none)
Required MISSING    : Accounting, Excel, Financial Reporting, GAAP
Preferred matched   : (none)
Top recommended edits (estimated score gain):
+ Accounting [required] +5.7 pts
+ Excel      [required] +5.7 pts
+ Financial Reporting [required] +5.7 pts
+ GAAP       [required] +5.7 pts
+ Auditing   [preferred] +2.9 pts

JOB: Data Analyst
Compatibility score : 47.4/100
Skill coverage      : 14% (cosine 0.045, beats 94% of corpus)
Semantic similarity : 0.443 (pre-trained Sentence-BERT embedding cosine)
Required matched    : SQL (0.91)
Required MISSING    : Data Analysis, Excel, Pandas, Python
Preferred matched   : Communication (0.50)
Top recommended edits (estimated score gain):
+ Data Analysis [required] +5.0 pts
+ Excel         [required] +5.0 pts
+ Pandas        [required] +5.0 pts
+ Python        [required] +5.0 pts
+ ETL           [preferred] +2.5 pts

JOB: Software Engineer - Backend
Compatibility score : 71.6/100
Skill coverage      : 53% (cosine 0.174, beats 100% of corpus)
Semantic similarity : 0.681 (pre-trained Sentence-BERT embedding cosine)
Required matched    : AWS (0.55), Agile (0.73), Git (0.80), Java (0.81), REST APIs (0.65), SQL (0.91)
Required MISSING    : Docker, Python
Preferred matched   : CI/CD (0.76), Linux (0.82), React (0.63)
Top recommended edits (estimated score gain):
+ Docker [required] +3.8 pts
+ Python [required] +3.8 pts
+ Kubernetes [preferred] +1.9 pts
+ Machine Learning [preferred] +1.9 pts

```

Figure 5. Console output of the analyzer for the demo resume across all three postings, including per-skill confidence and ranked edit recommendations.

8. Web Interface

A Streamlit app (app.py) exposes the full pipeline to an end user: upload or paste a resume, pick or paste a job description, and see the compatibility score with its three-way breakdown, the Naive Bayes predicted category, matched vs. missing skills with confidence, and the ranked recommended edits. The corpus, matcher, and classifier load once per session and are cached.

This was verified live end to end: uploading a resume file through the browser correctly triggers ingestion, confidence-weighted extraction, and the blended score, matching the command-line output for the same input.

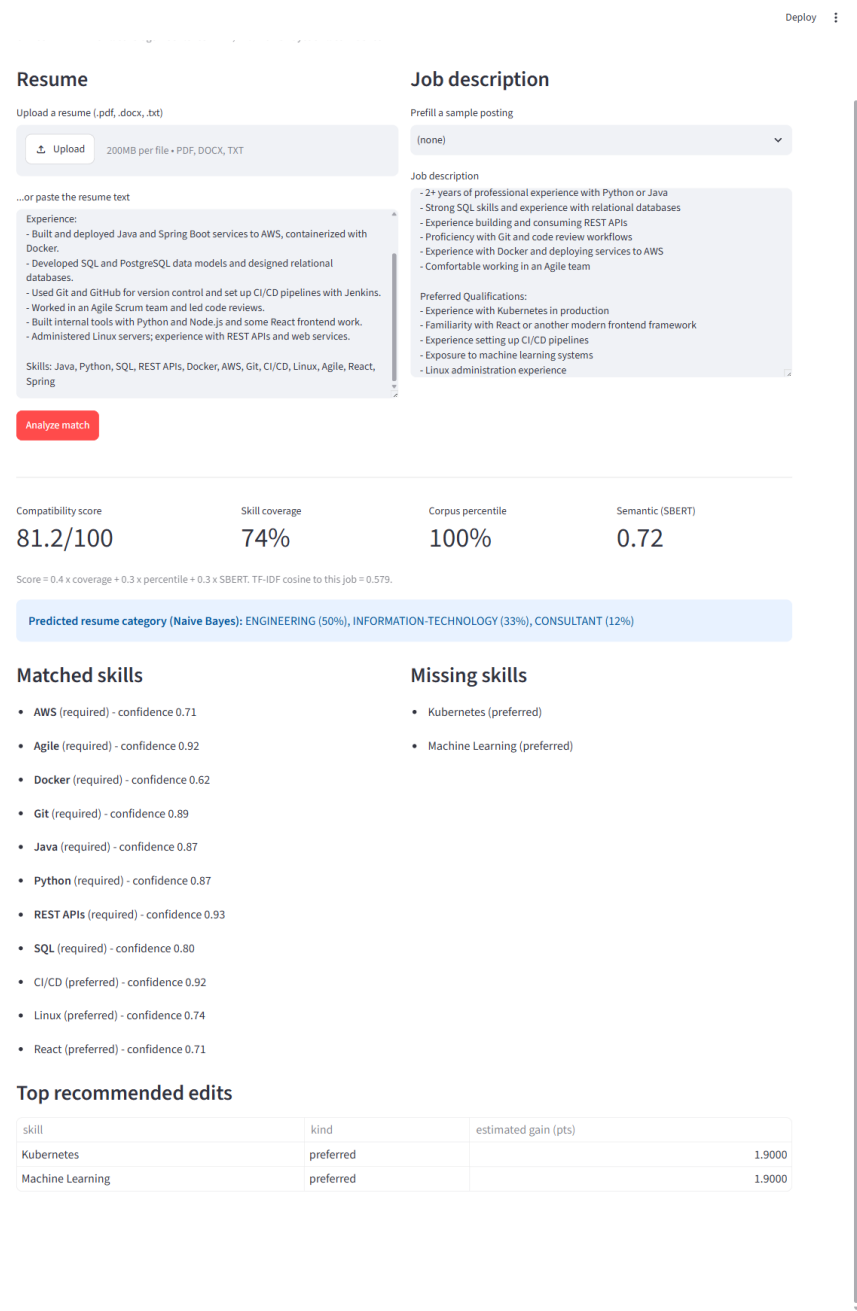


Figure 6. The web interface after analyzing a resume against the backend software engineer posting: score breakdown, predicted category, and matched/missing skills with confidence.

9. Challenges

- **Class imbalance in the dataset.** The largest categories have 120 resumes while the smallest has only 22. Addressed with a stratified train/test split and macro F1 alongside raw accuracy.

- **Raw cosine similarities are not interpretable.** Resume-to-posting TF-IDF cosine values range from about 0.01 to 0.5, so presenting them directly would make most matches look weak. Solved with corpus-percentile calibration: the score reports where the resume ranks against all resumes in the corpus for that same posting.
- **Lexical overlap misses genuine matches.** TF-IDF alone gives no credit to paraphrased skills. Adding a pre-trained Sentence-BERT similarity term as a third, independent signal closes this gap without requiring any model training.
- **Trusting a skill mention vs. a skill list.** A skill named once in a flat 'Skills:' section is weaker evidence than one used in context ('built X with Y') or one consistent with the resume's overall predicted domain. The three-signal confidence model (frequency, context, Naive Bayes category agreement) addresses this directly.
- **Reproducibility under real classifier training.** Naive Bayes is exactly reproducible run to run; Logistic Regression shows small drift (under 0.3 points) from solver-level floating-point non-determinism even with a fixed random seed, documented rather than hidden.

10. Conclusion

The Resume Analyzer and Job Matcher is a complete, working prototype of every stage proposed at the start of the project: multi-format resume ingestion, confidence-aware skill extraction over an expanded, dataset-wide taxonomy, constraint-based job parsing, a three-signal compatibility score that blends lexical (TF-IDF), structured (skill coverage), and semantic (pre-trained Sentence-BERT) evidence, a greedy recommendation engine, a Naive Bayes category classifier, and a browser-based interface, verified end to end both from the command line and through the live web UI.

Team Contributions

Martin Dang (50%)	Tuan Khang Phan (50%)
Skill taxonomy design and alias normalization; skill extraction and confidence module; job description constraint parser; recommendation engine and ranking logic	Dataset acquisition and preprocessing; TF-IDF matching engine and percentile calibration; Sentence-BERT integration; PDF and DOCX ingestion; classifier experiments and evaluation; Streamlit web UI